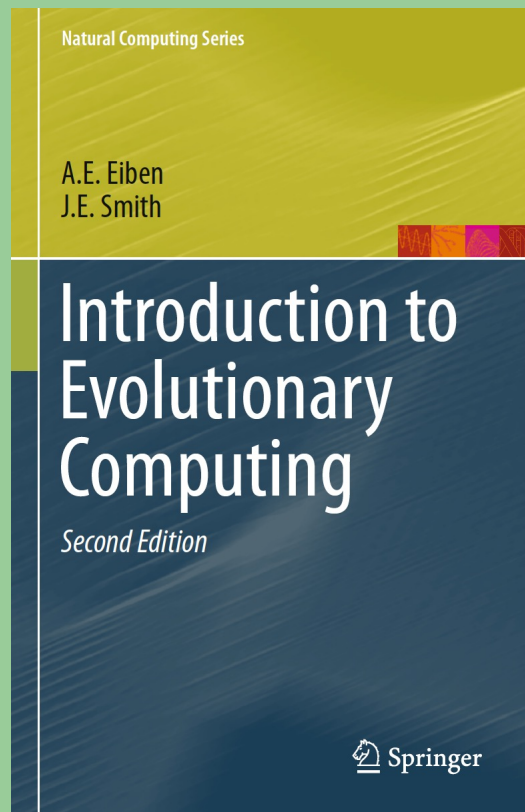


Introduction to Evolutionary Computing

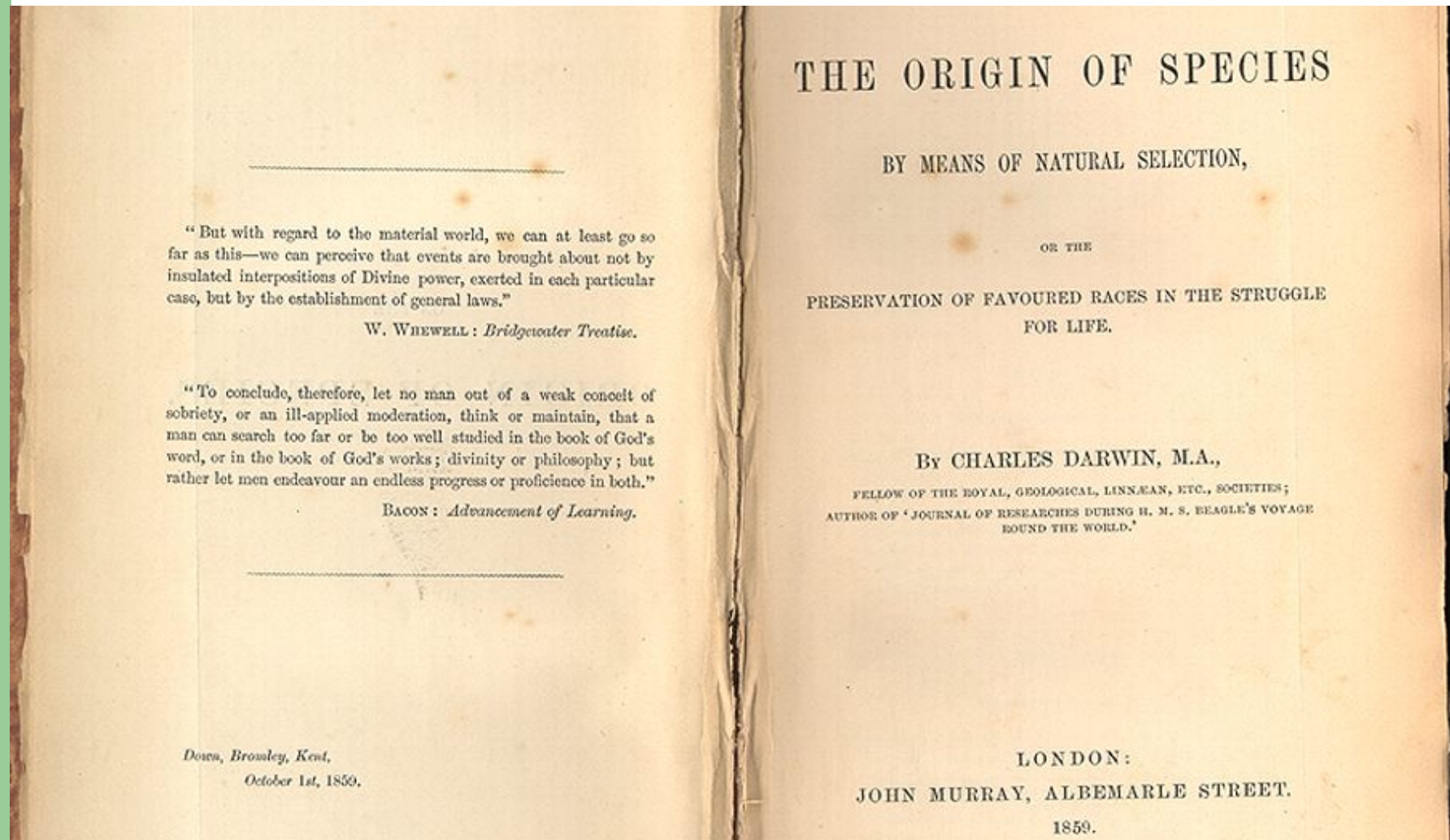


Adapted from
A.E. Eiben and J.E. Smith,
*Introduction to Evolutionary
Computing 2nd edition*,
Springer 2015

Introduction

Chapter 2

The Origin of Species by Charles Darwin



Darwinian Evolution: Survival of the fittest

- All environments have finite resources
(i.e., can only support a limited number of individuals)
- Lifeforms have basic instinct/ lifecycles geared towards reproduction
- Therefore some kind of selection is inevitable
- Those individuals that compete for the resources most effectively have increased chance of reproduction
- Note: fitness in natural evolution is a derived, secondary measure, i.e., we (humans) assign a high fitness to individuals with many offspring

Darwinian Evolution: Summary

- Population consists of diverse set of individuals
- Combinations of traits that are better adapted tend to increase representation in population

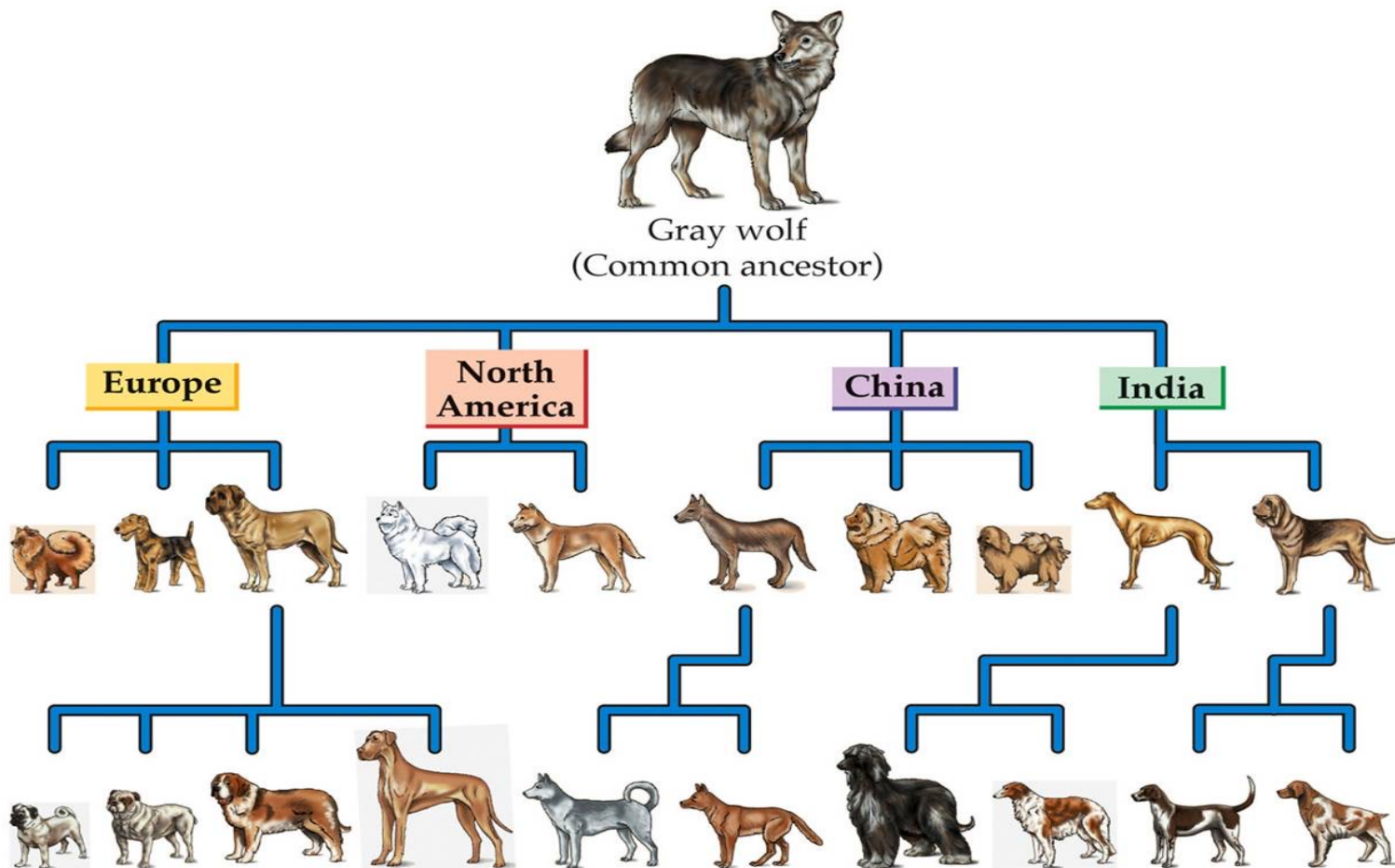
Individuals are “units of selection”

- Variations occur through random changes yielding constant source of diversity, coupled with selection means that:

Population is the “unit of evolution”

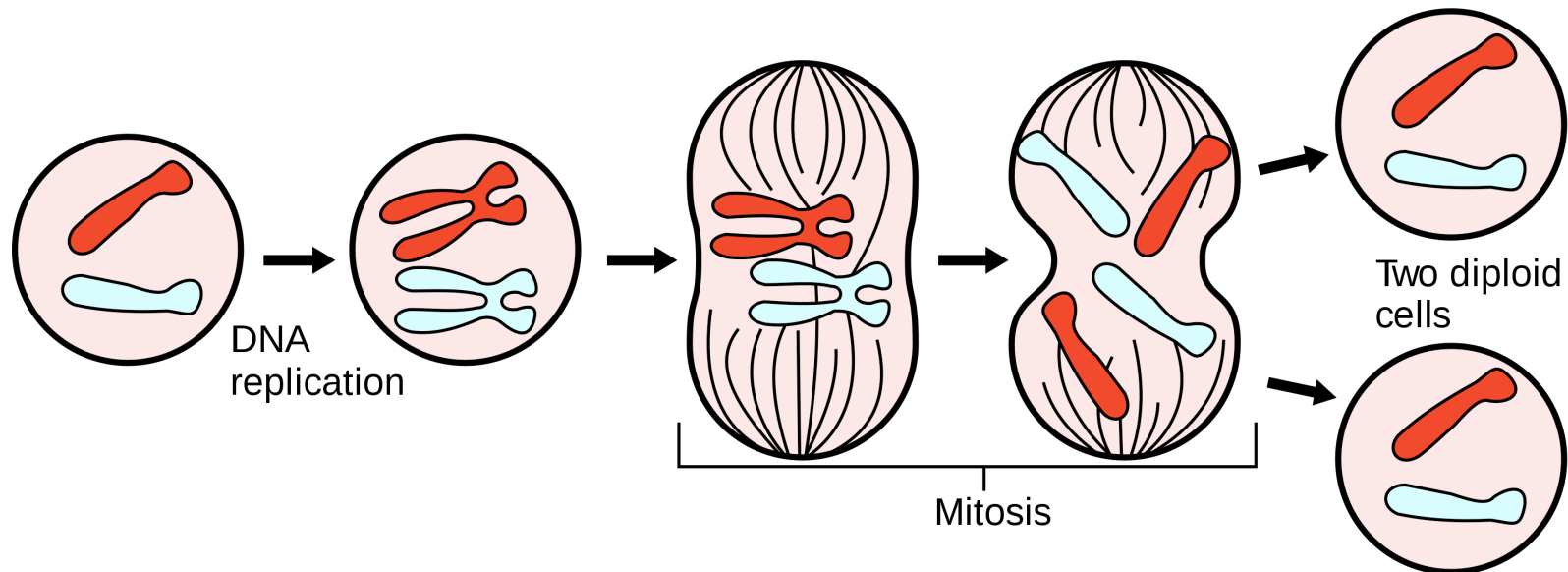
- Note the absence of “guiding force”

Dogs as Artificial Evolution



Ordinary cell division: mitosis

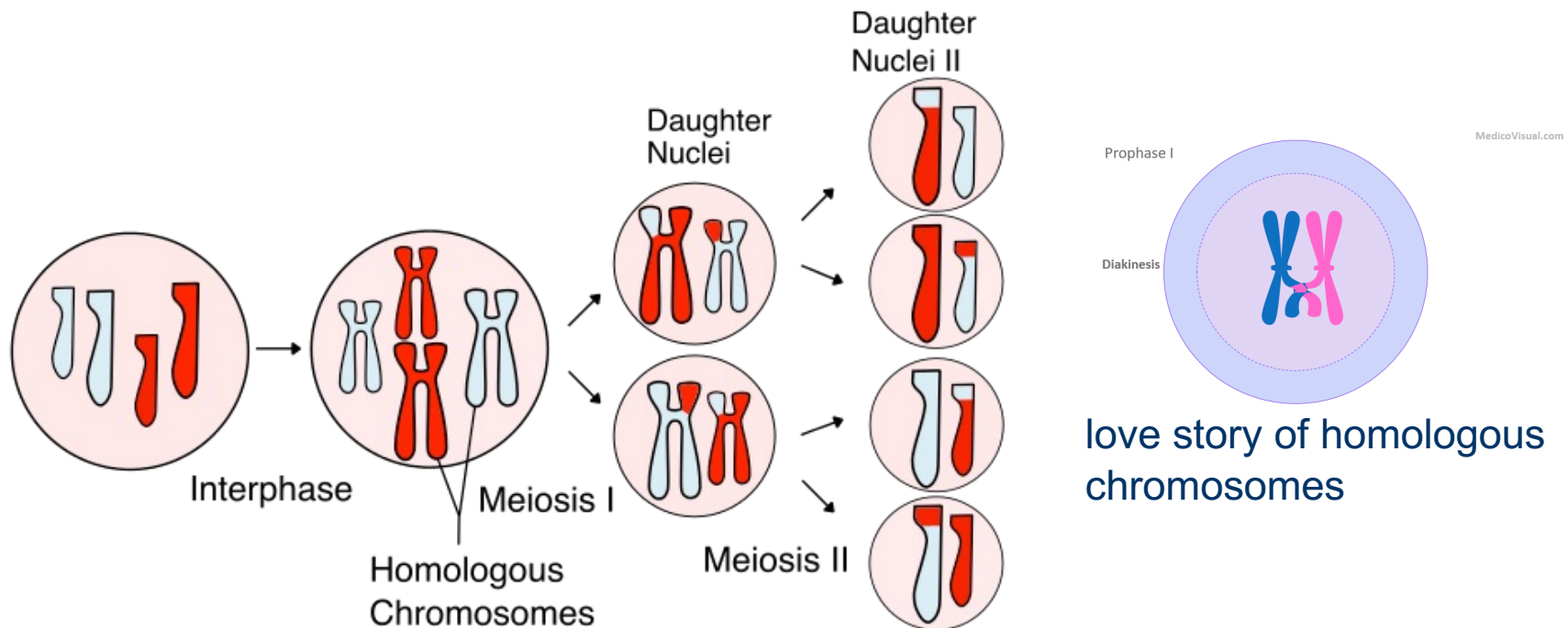
- Mitosis divides the chromosomes in a cell nucleus
- Mitosis gives rise to genetically identical cells
- The number of chromosomes is maintained



From wikipedia: mitosis

Crossing-over during meiosis

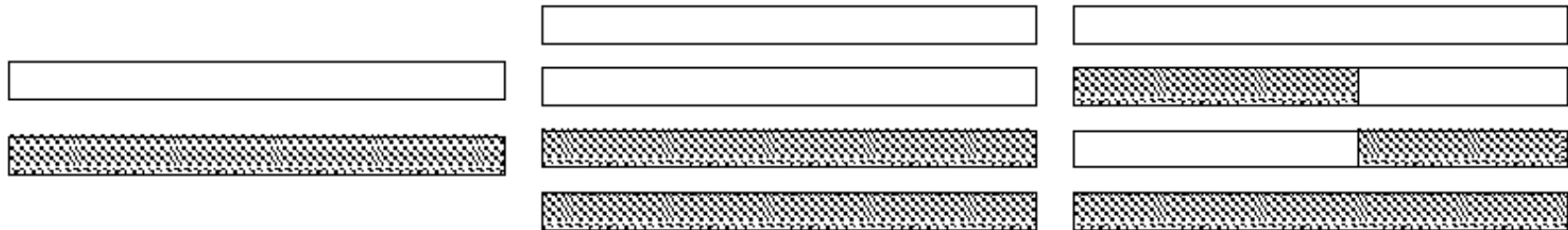
- Chromosome pairs align and duplicate
- Homologous chromosomes swap parts of themselves
- Cells divide again to form haploid gametes



From wikipedia: meiosis

Crossing-over during meiosis

- Chromosome pairs align and duplicate
- Inner pairs link at a *centromere* and swap parts of themselves



- Outcome is one copy of maternal/paternal chromosome plus two entirely new combinations
- After crossing-over one of each pair goes into each gamete

Mutation

- Occasionally some of the genetic material changes very slightly during this process (replication error)
- This means that the child might have genetic material information not inherited from either parent
- This can be
 - catastrophic: offspring is not viable (most likely)
 - neutral: new feature does not influence fitness
 - advantageous: strong new feature occurs
- Redundancy in the genetic code forms a good way of error checking

Motivations for EC: 1

- Nature has always served as a source of inspiration for engineers and scientists
- The best problem solver known in nature is:
 - **the (human) brain** that created “the wheel, New York, wars and so on” (after Douglas Adams’ Hitch-Hikers Guide)
 - **the evolution mechanism** that created the human brain (after Darwin’s Origin of Species)
- Answer 1 → neurocomputing
- Answer 2 → evolutionary computing

What is an Evolutionary Algorithm?

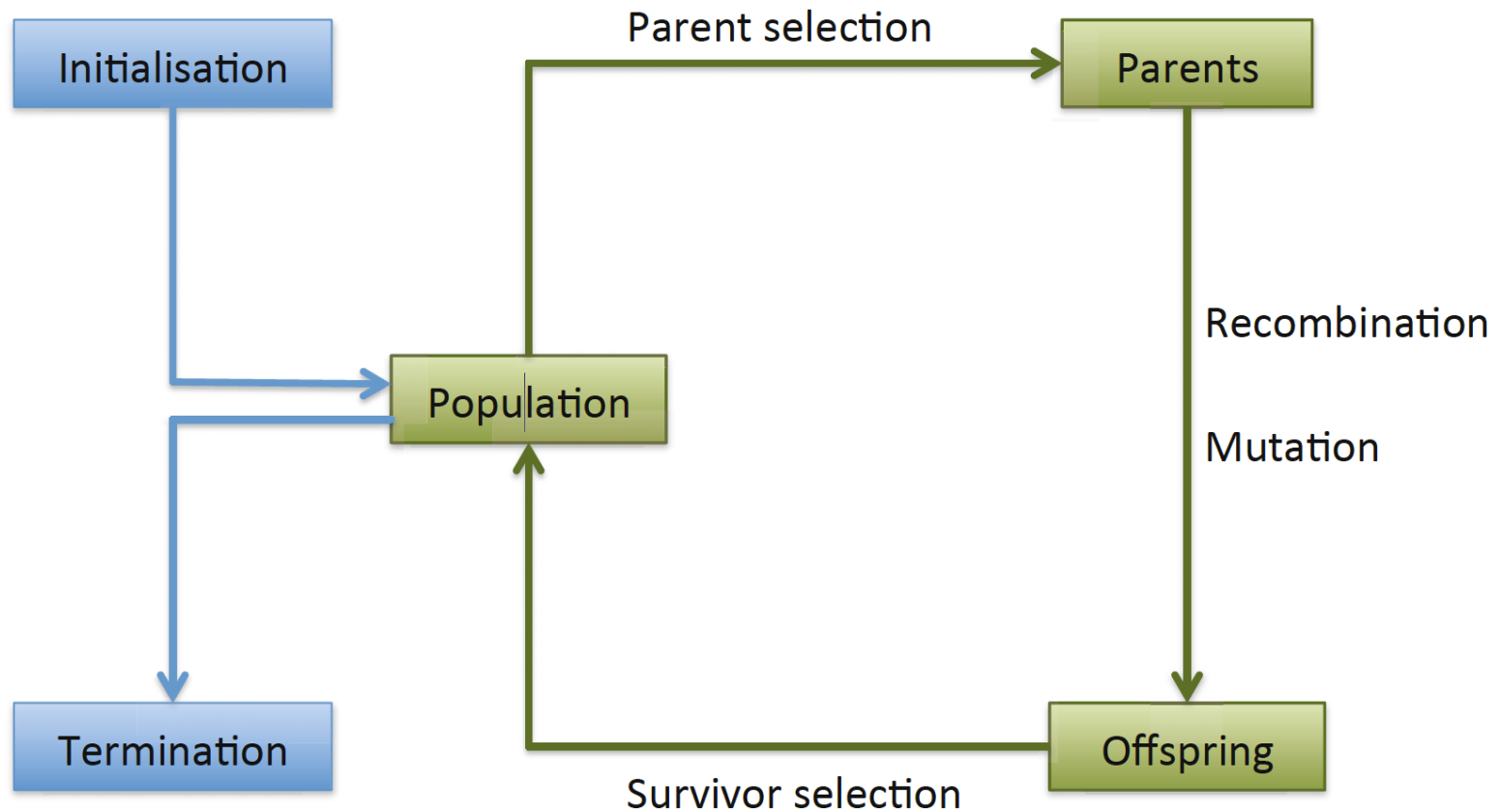
Chapter 3



Recap of EC metaphor

- A population of individuals exists in an environment with limited resources
- **Competition** for those resources causes selection of those **fitter** individuals that are better adapted to the environment
- These individuals act as seeds for the generation of new individuals through recombination and mutation
- The new individuals have their fitness evaluated and compete (possibly also with parents) for survival.
- Over time **Natural selection** causes a rise in the fitness of the population

General Scheme of EAs



Pseudo-code for typical EA

```
BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
```


What are the different types of EAs

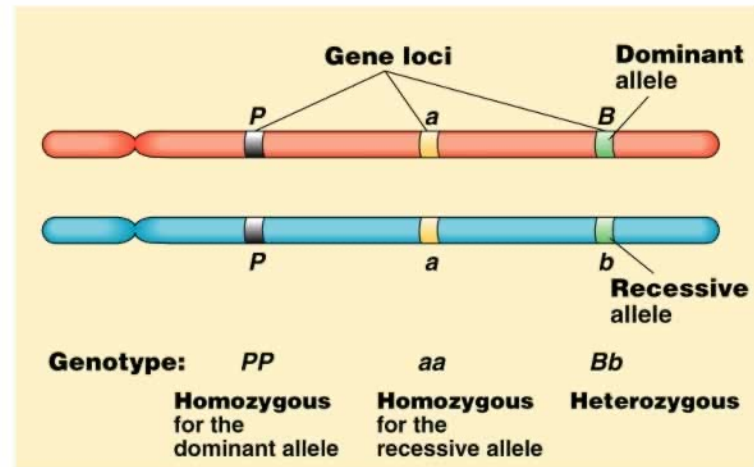
- Historically different flavours of EAs have been associated with different representations
 - Binary strings : Genetic Algorithms
 - Real-valued vectors : Evolution Strategies
 - Finite state Machines: Evolutionary Programming
 - Trees: Genetic Programming
- These differences are largely irrelevant, best strategy
 - choose representation to suit problem
 - choose variation operators to suit representation
- Selection operators only use fitness and so are independent of representation

Representations

- Candidate solutions (**individuals**) exist in *phenotype* space
- They are encoded in **chromosomes**, which exist in *genotype* space
 - Encoding : phenotype=> genotype (not necessarily one to one)
 - Decoding : genotype=> phenotype (must be one to one)
- Chromosomes contain **genes**, which are in (usually fixed) positions called **loci** (sing. locus) and have a value (**allele**)

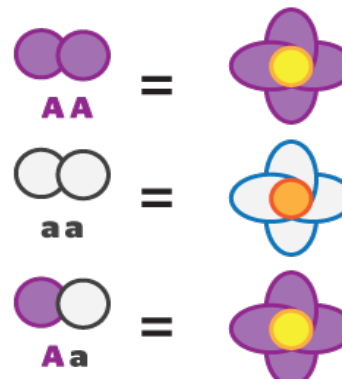
In order to find the global optimum, every feasible solution must be represented in genotype space

Representations



©Addison Wesley Longman, Inc.

 = is the dominant allele for flower colour
 = is the recessive allele for flower colour



Evaluation (Fitness) Function

- Represents the requirements that the population should adapt to
- a.k.a. *quality* function or *objective* function
- Assigns a single real-valued fitness to each phenotype which forms the basis for selection
 - So the more discrimination (different values) the better
- Typically we talk about fitness being maximised
 - Some problems may be best posed as minimisation problems, but conversion is trivial

Population

- Holds (representations of) possible solutions
- Usually has a fixed size and is a *multiset* of genotypes
- Some sophisticated EAs also assert a spatial structure on the population e.g., a grid.
- Selection operators usually take whole population into account i.e., reproductive probabilities are *relative* to *current* generation
- **Diversity** of a population refers to the number of different fitnesses / phenotypes / genotypes present (note not the same thing)

Parent Selection Mechanism

- Assigns variable probabilities of individuals acting as parents depending on their fitnesses
- Usually probabilistic
 - high quality solutions more likely to become parents than low quality
 - but not guaranteed
 - even worst in current population usually has non-zero probability of becoming a parent
- This *stochastic* nature can aid escape from local optima

Variation Operators

- Role is to generate new candidate solutions
- Usually divided into two types according to their **arity** (number of inputs):
 - Arity 1 : mutation operators
 - Arity >1 : Recombination operators
 - Arity = 2 typically called **crossover**
- There has been much debate about relative importance of recombination and mutation
 - Nowadays most EAs use both
 - Choice of particular variation operators is representation dependant

Mutation

- Acts on one genotype and delivers another
- Element of randomness is essential and differentiates it from other unary heuristic operators
- Importance ascribed depends on representation and dialect:
 - Binary GAs – background operator responsible for preserving and introducing diversity
 - EP for FSMs - continuous variables – only variation operator
 - GP – hardly used
- May guarantee connectedness of search space and hence convergence proofs

Recombination

- Merges information from parents into offspring
- Choice of what information to merge is stochastic
- Most offspring may be worse, or the same as the parents
- Hope is that some are better by combining elements of genotypes that lead to good traits
- Principle has been used for millennia by breeders of plants and livestock

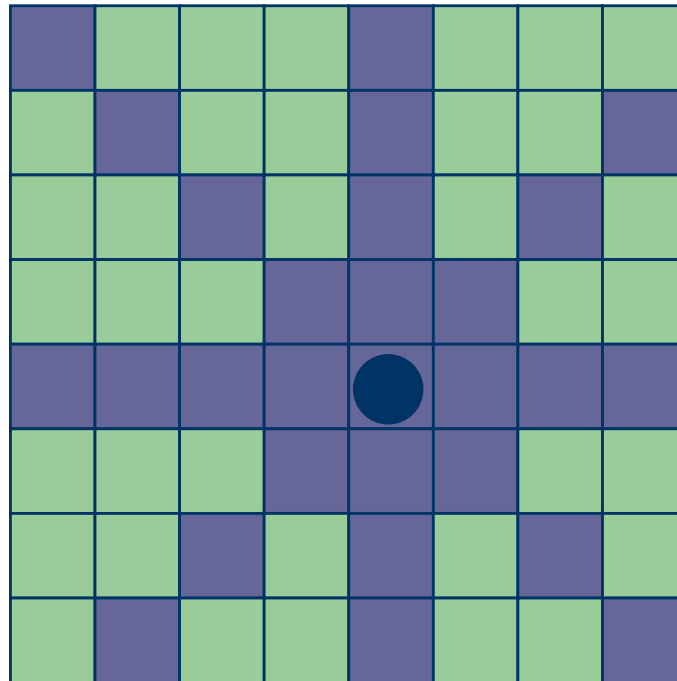
Survivor Selection

- a.k.a. *replacement*
- Most EAs use fixed population size so need a way of going from (parents + offspring) to next generation
- Often deterministic
 - Fitness based : e.g., rank parents+offspring and take best
 - Age based: make as many offspring as parents and delete all parents
- Sometimes do combination (elitism)

Initialisation / Termination

- Initialisation usually done at random,
 - Need to ensure even spread and mixture of possible allele values
 - Can include existing solutions, or use problem-specific heuristics, to “seed” the population
- Termination condition checked every generation
 - Reaching some (known/hoped for) fitness
 - Reaching some maximum allowed number of generations
 - Reaching some minimum level of diversity
 - Reaching some specified number of generations without fitness improvement

Example: the 8 queens problem

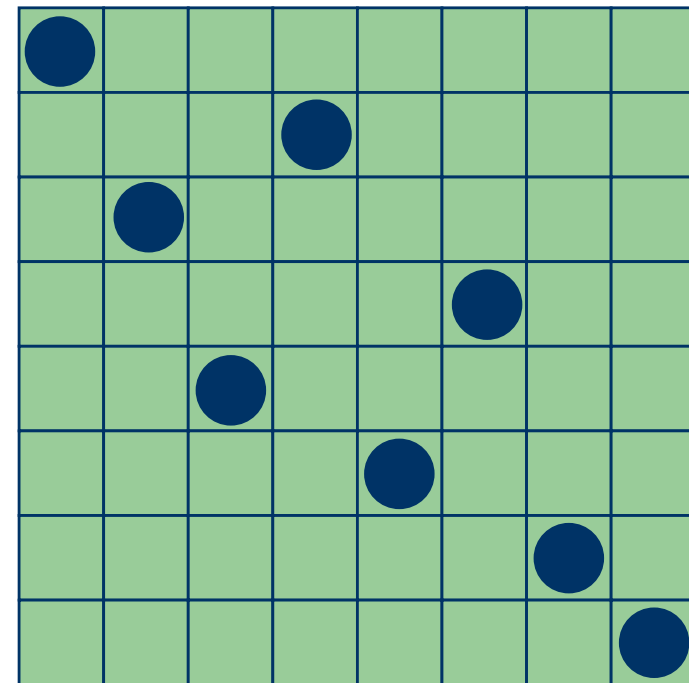


Place 8 queens on an 8x8 chessboard in such a way that they cannot check each other

The 8 queens problem: representation

Phenotype:
a board configuration

Genotype:
a permutation of
the numbers 1 - 8



Obvious mapping

1	3	5	2	6	4	7	8
---	---	---	---	---	---	---	---

8 Queens Problem: Fitness evaluation

- Penalty of one queen:
the number of queens she can check.
- Penalty of a configuration:
the sum of the penalties of all queens.
- Note: penalty is to be minimized
- Fitness of a configuration:
inverse penalty to be maximized

The 8 queens problem: Mutation

Small variation in one permutation, e.g.:

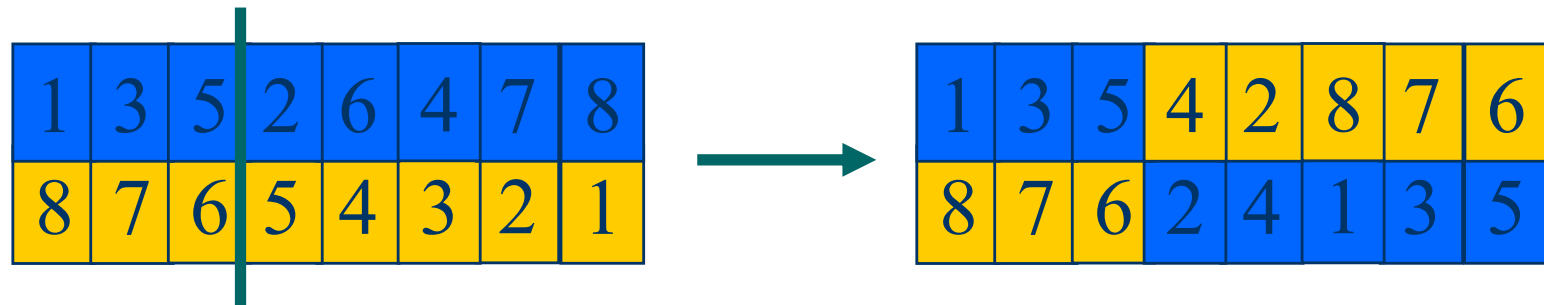
- swapping values of two randomly chosen positions,



The 8 queens problem: Recombination

Combining two permutations into two new permutations:

- choose random crossover point
- copy first parts into children
- create second part by inserting values from other parent:
 - in the order they appear there
 - beginning after crossover point
 - skipping values already in child



The 8 queens problem: Selection

- Parent selection:
 - Pick 5 parents and take best two to undergo crossover
- Survivor selection (replacement)

Checks which old individuals should be deleted to make place for the new ones – provided the new ones are better:

 - merge the population and offspring
 - rank them according to fitness
 - delete the worst two

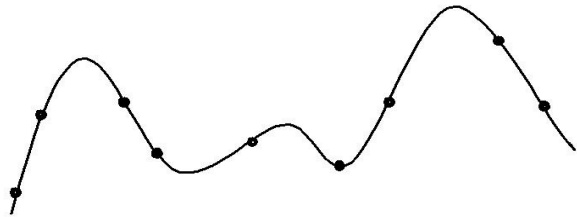
8 Queens Problem: summary

Representation	Permutations
Recombination	“Cut-and-crossfill” crossover
Recombination probability	100%
Mutation	Swap
Mutation probability	80%
Parent selection	Best 2 out of random 5
Survival selection	Replace worst
Population size	100
Number of Offspring	2
Initialisation	Random
Termination condition	Solution or 10,000 fitness evaluation

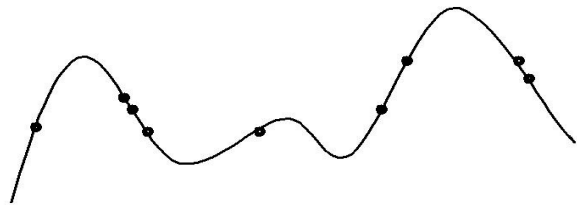
Note that this is ***only one possible*** set of choices of operators and parameters

Typical behaviour of an EA

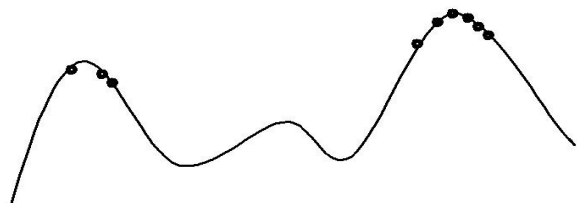
Phases in optimising on a 1-dimensional fitness landscape



Early phase:
quasi-random population distribution



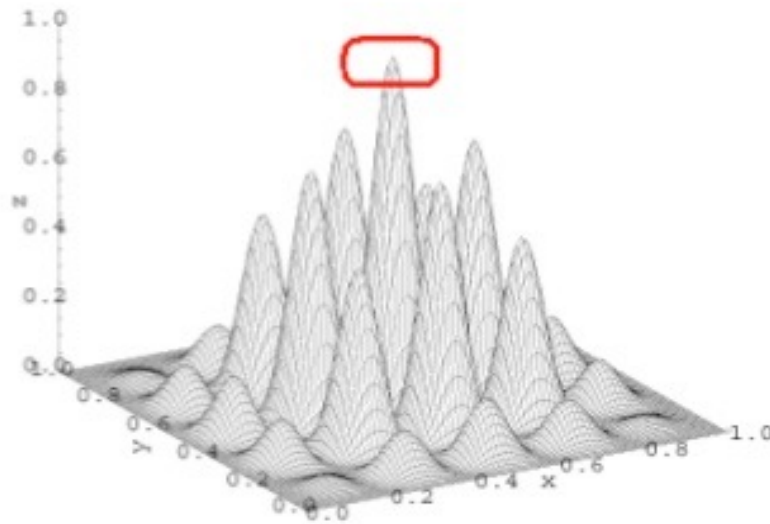
Mid-phase:
population arranged around/on hills



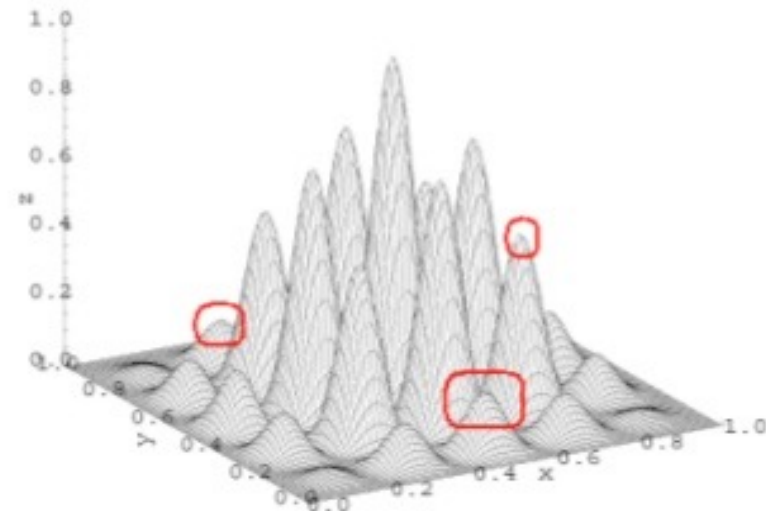
Late phase:
population concentrated on high hills

Typical behaviour of an EA

Risk: Premature convergence

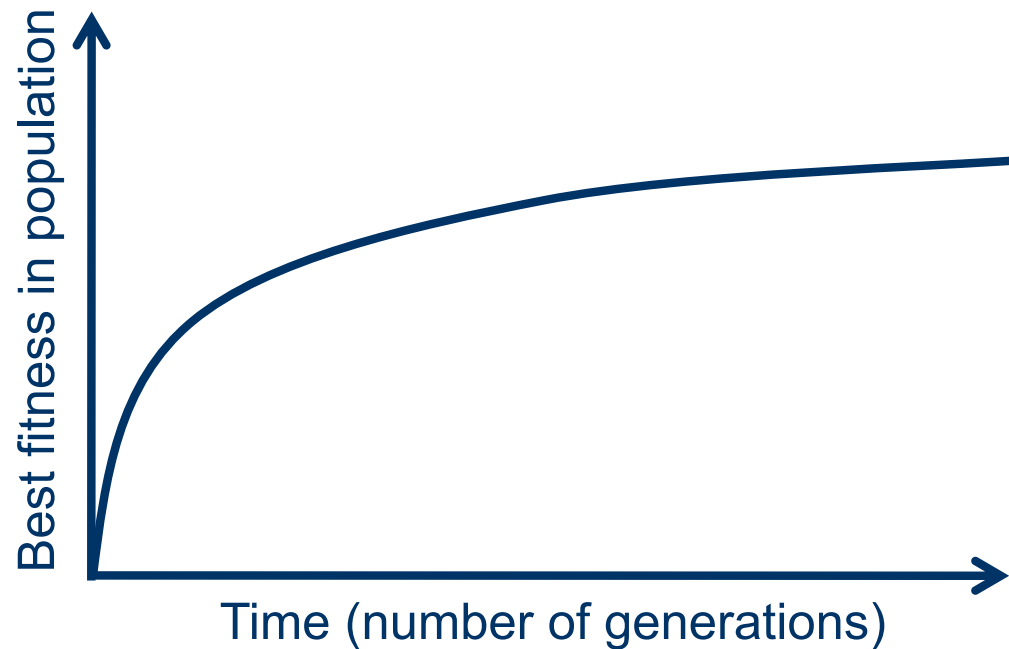


Our objective is to attain some maximum value



Premature convergence at certain local niches

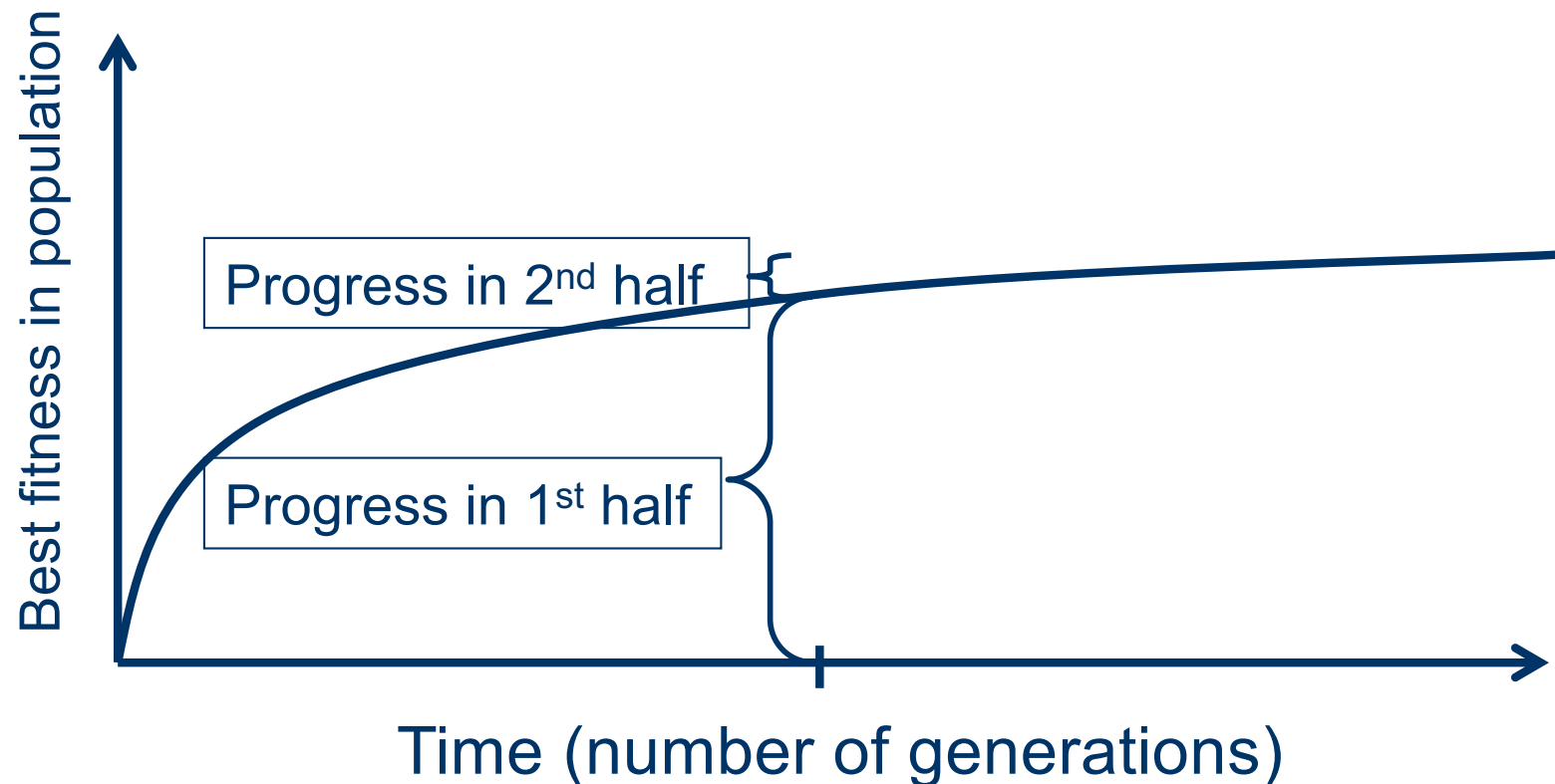
Typical run: progression of fitness



Typical run of an EA shows so-called “anytime behavior”:

- the search can be stopped at any time
- the algorithm will have some solution, even if it is suboptimal

Are long runs beneficial?



- Answer:
 - it depends on how much you want the last bit of progress
 - it may be better to do more shorter runs

Genetic Algorithms

Chapter 3



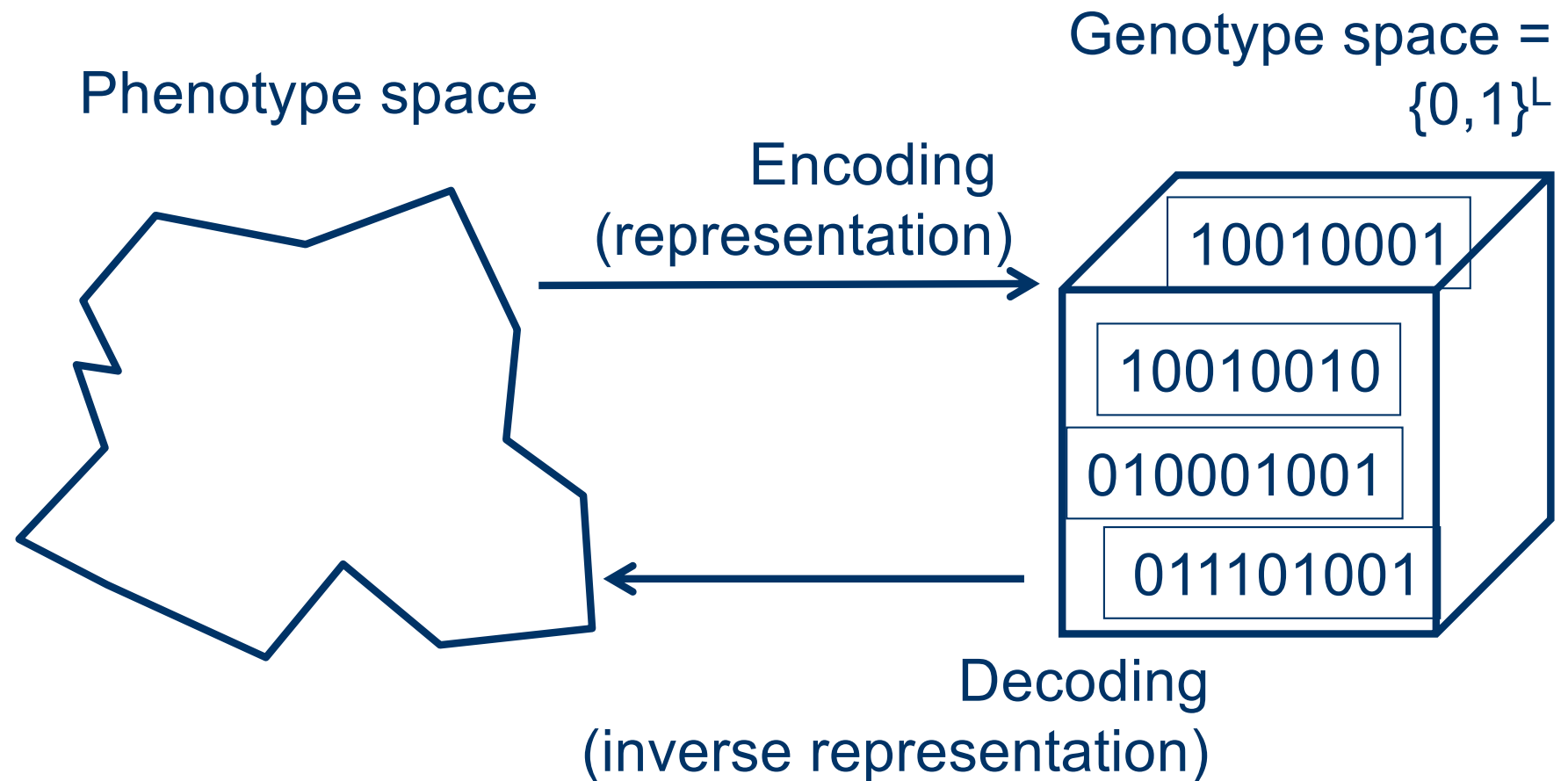
Genetic algorithms

- Holland's original GA is now known as the simple genetic algorithm (SGA)
- Other GAs use different:
 - Representations
 - Mutations
 - Crossovers
 - Selection mechanisms

SGA technical summary tableau

Representation	Binary strings
Recombination	N-point crossover or uniform
Mutation	Bitwise bit-flipping with fixed probability
Parent selection	Fitness-Proportionate (Roulette Wheel)
Survivor selection	Generational (All children replace parents)
Speciality	Emphasis on crossover

Representation



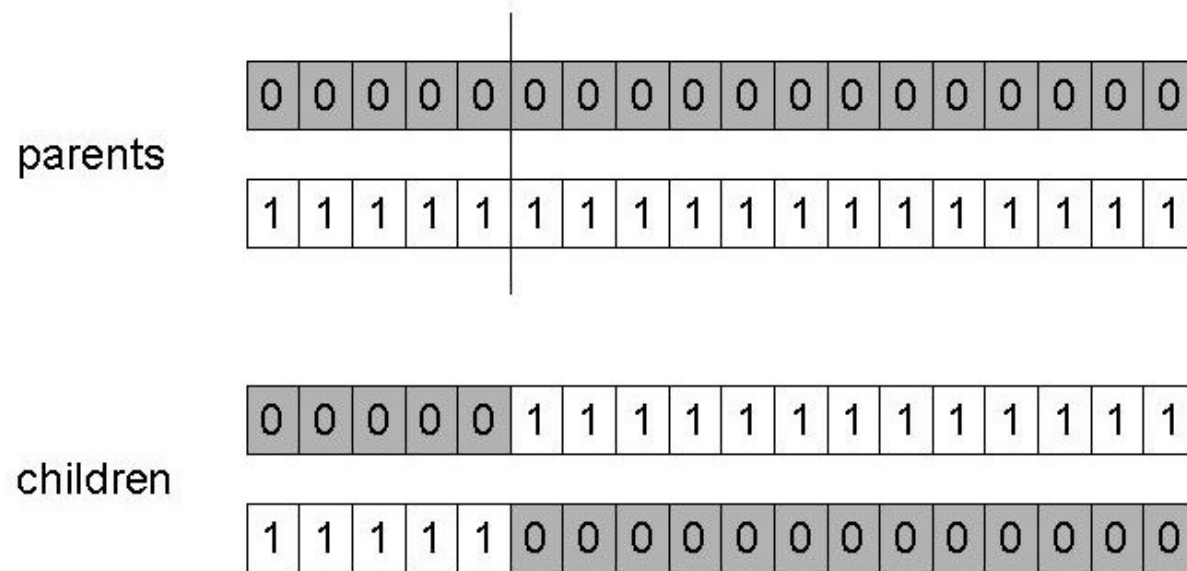
SGA reproduction cycle

1. Select parents for the mating pool
(size of mating pool* = population size)
2. Shuffle the mating pool
3. For each consecutive pair apply crossover with probability p_c , otherwise copy parents
4. For each offspring apply mutation (bit-flip with probability p_m independently for each bit)
5. Replace the whole population with the resulting offspring

* Mating pool not equal to population; some parents are repeated

SGA operators: 1-point crossover

- Choose a random point on the two parents
- Split parents at this crossover point
- Create children by exchanging tails
- P_c typically in range (0.6, 0.9)



SGA operators: mutation

- Alter each gene independently with a probability p_m
- p_m is called the mutation rate
 - Typically between $1/\text{pop_size}$ and $1/\text{chromosome_length}$

parent

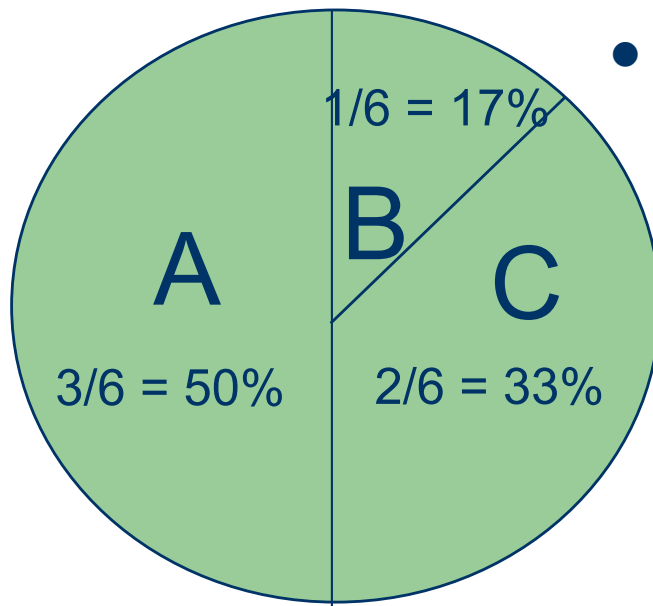
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

child

0	1	0	0	1	0	1	1	0	0	0	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

SGA operators: Selection (parents)

- Main idea: better individuals get higher chance
 - Chances proportional to fitness
 - Implementation: roulette wheel technique
 - Assign to each individual a part of the roulette wheel
 - Spin the wheel n times to select n individuals



fitness(A) = 3

fitness(B) = 1

fitness(C) = 2

An example after Goldberg '89

- Simple problem: $\max x^2$ over $\{0,1,\dots,31\}$
- GA approach:
 - Representation: binary code, e.g. $01101 \leftrightarrow 13$
 - Population size: 4
 - 1-point xover, bitwise mutation
 - Roulette wheel selection
 - Random initialisation
- We show one generational cycle done by hand

x^2 example: selection

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$
1	0 1 1 0 1	13	169	0.14
2	1 1 0 0 0	24	576	0.49
3	0 1 0 0 0	8	64	0.06
4	1 0 0 1 1	19	361	0.31
Sum			1170	1.00
Average			293	0.25
Max			576	0.49

X² example: crossover

String no.	Mating pool	Crossover point	Offspring after xover	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 1	4	0 1 1 0 0	12	144
2	1 1 0 0 0	4	1 1 0 0 1	25	625
2	1 1 0 0 0	2	1 1 0 1 1	27	729
4	1 0 0 1 1	2	1 0 0 0 0	16	256
Sum					1754
Average					439
Max					729

X^2 example: mutation

String no.	Offspring after xover	Offspring after mutation	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 0	1 1 1 0 0	28	784
2	1 1 0 0 1	1 1 0 0 1	25	625
2	1 1 0 1 1	1 1 0 1 1	27	729
4	1 0 0 0 0	1 0 1 0 0	20	400
Sum				2538
Average				634.5
Max				784

Components of a GA

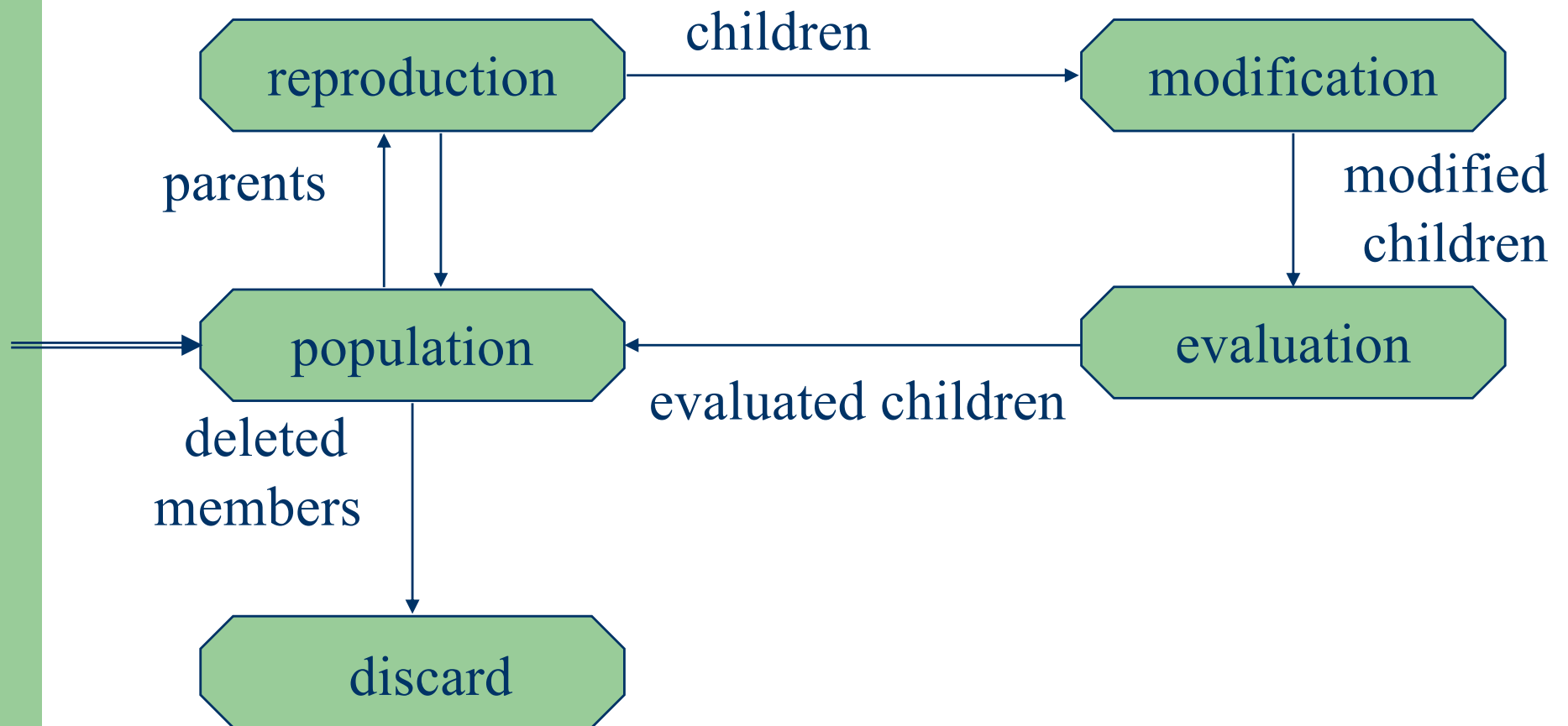
A problem to solve, and ...

- Encoding technique *(gene, chromosome)*
- Initialization procedure *(creation)*
- Evaluation function *(environment)*
- Selection of parents *(reproduction)*
- Genetic operators *(mutation, recombination)*
- Parameter settings *(practice and art)*

Simple Genetic Algorithm

```
{  
    initialize population;  
    evaluate population;  
    while TerminationCriteriaNotSatisfied  
    {  
        select parents for reproduction;  
        perform recombination and mutation;  
        evaluate population;  
    }  
}
```

The GA Cycle of Reproduction



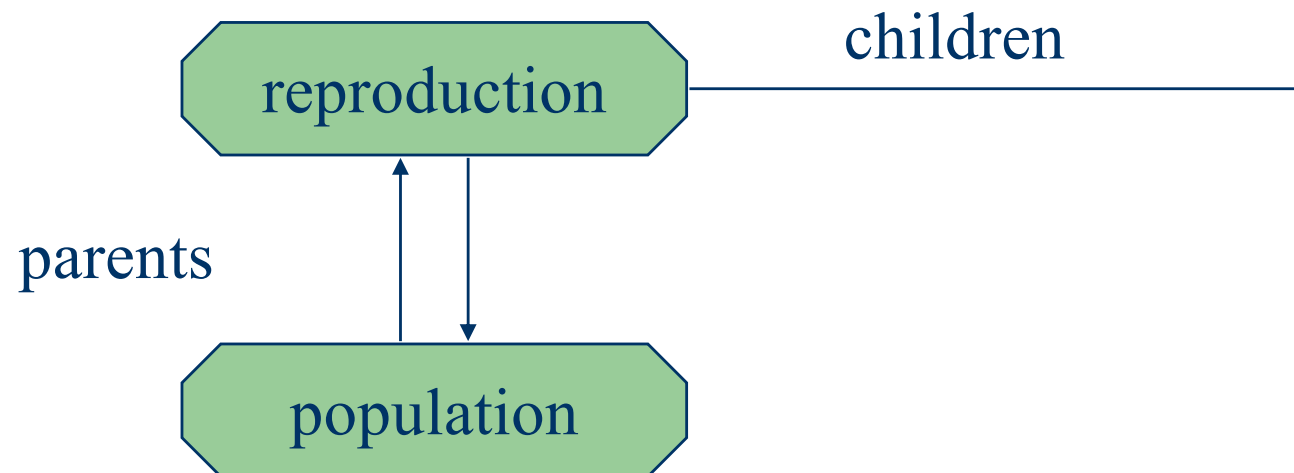
Population



Chromosomes could be:

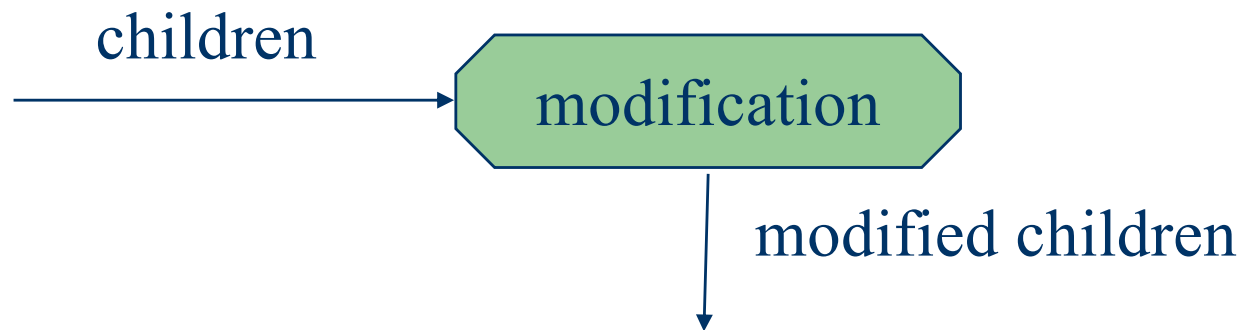
- Bit strings (0101 ... 1100)
- Real numbers (43.2 -33.1 ... 0.0 89.2)
- Permutations of element (E11 E3 E7 ... E1 E15)
- Lists of rules (R1 R2 R3 ... R22 R23)
- Program elements (genetic programming)
- ... any data structure ...

Reproduction



Parents are selected at random with selection chances biased in relation to chromosome evaluations.

Chromosome Modification



- Modifications are stochastically triggered
- Operator types are:
 - Mutation
 - Crossover (recombination)

Mutation: Local Modification

Before: (1 0 1 1 0 1 1 0)
After: (1 0 1 0 0 1 1 0)

Before: (1.38 -69.4 326.44 0.1)
After*: (1.38 -67.5 326.44 0.1)

- Causes movement in the search space (local or global)
- Restores lost information to the population

* e.g. A Gaussian Mutator performs the mutation of a numeric gene. This mutator picks a new value based on a Gaussian distribution around the current value of the gene.

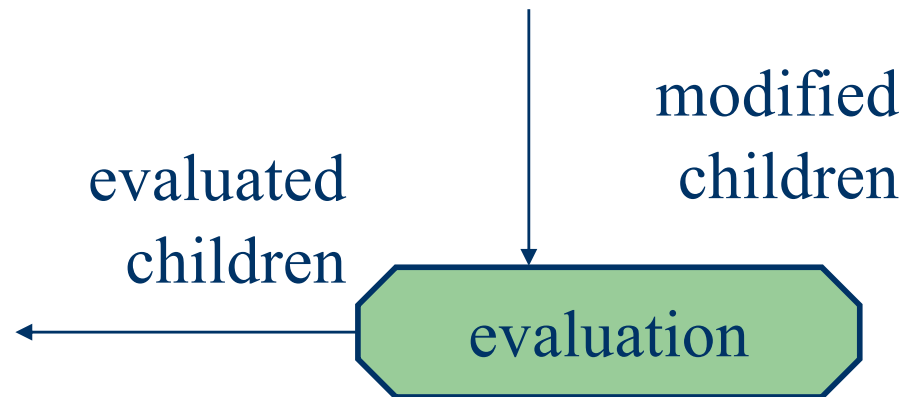
Crossover: Recombination

P1	(0 1 1 0 1 0 0 0)	(0 1 0 0 1 0 0 0)	C1
P2	(1 1 0 1 1 0 1 0)	(1 1 1 1 1 0 1 0)	C2

Crossover is a critical feature of genetic algorithms:

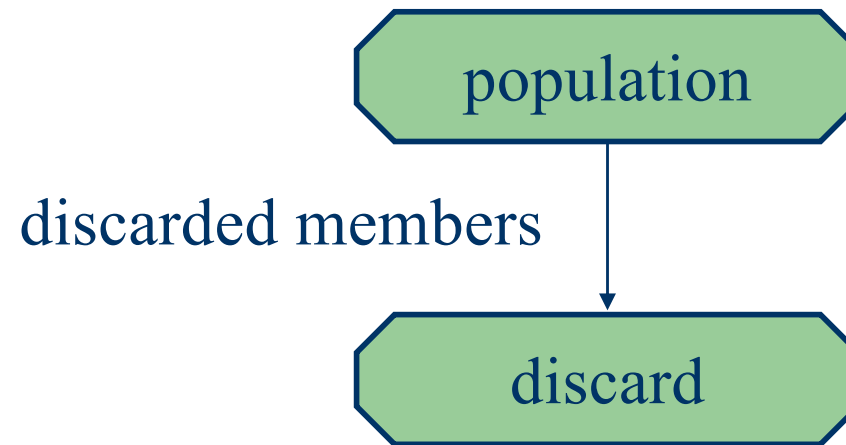
- It greatly accelerates search early in evolution of a population
- It leads to effective combination of schemata (subsolutions on different chromosomes)

Evaluation



- The evaluator decodes a chromosome and assigns it a fitness measure
- The evaluator is the only link between a classical GA and the problem it is solving

Deletion



- *Generational GA*:
entire populations replaced with each iteration
- *Steady-state GA*:
a few members replaced each generation

Issues for GA Practitioners

- Choosing basic implementation issues:
 - representation
 - population size, mutation rate, ...
 - selection, deletion policies
 - crossover, mutation operators
- Termination Criteria
- Performance, scalability
- Solution is only as good as the evaluation function (often hardest part)

Benefits of Genetic Algorithms

- Concept is easy to understand
- Modular, separate from application
- Supports multi-objective optimization
- Good for “noisy” environments
- Always an answer; answer gets better with time
- Inherently parallel; easily distributed

Benefits of Genetic Algorithms (cont.)

- Many ways to speed up and improve a GA-based application as knowledge about problem domain is gained
- Easy to exploit previous or alternate solutions
- Flexible building blocks for hybrid applications
- Substantial history and range of use

When to Use a GA

- Alternate solutions are too slow or overly complicated
- Need an exploratory tool to examine new approaches
- Problem is similar to one that has already been successfully solved by using a GA
- Want to hybridize with an existing solution
- Benefits of the GA technology meet key problem requirements

A Simple Example

The Traveling Salesman Problem:

Find a tour of a given set of cities so that

- each city is visited only once
- the total distance traveled is minimized

Representation

Representation is an ordered list of city numbers known as an *order-based GA*.

1) London	3) Dunedin	5) Beijing	7) Tokyo
2) Venice	4) Singapore	6) Phoenix	8) Victoria

CityList1 (3 5 7 2 1 6 4 8)

CityList2 (2 5 7 6 8 1 3 4)

Crossover

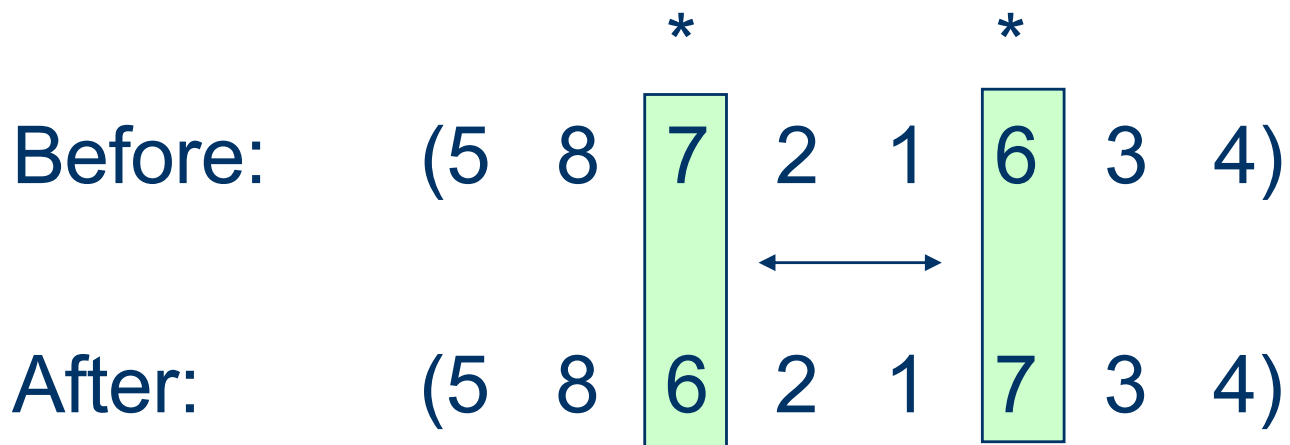
Crossover combines inversion and recombination:

			*		*				
Parent1	(	3	5	7	2	1	6	4	8)
Parent2	(	2	5	7	6	8	1	3	4)
Child	(	5	8	7	2	1	6	3	4)

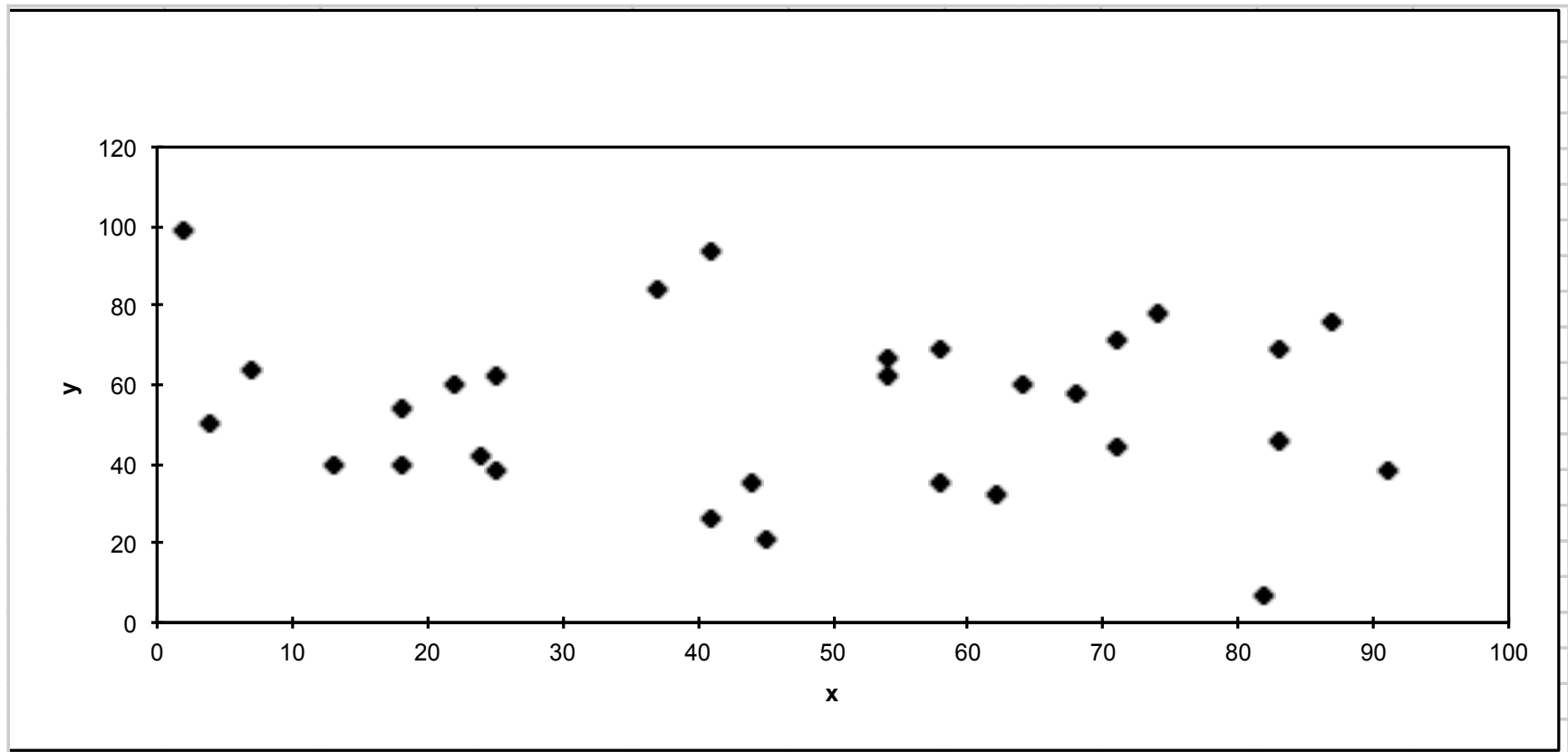
This operator is called the *Order1* crossover.

Mutation

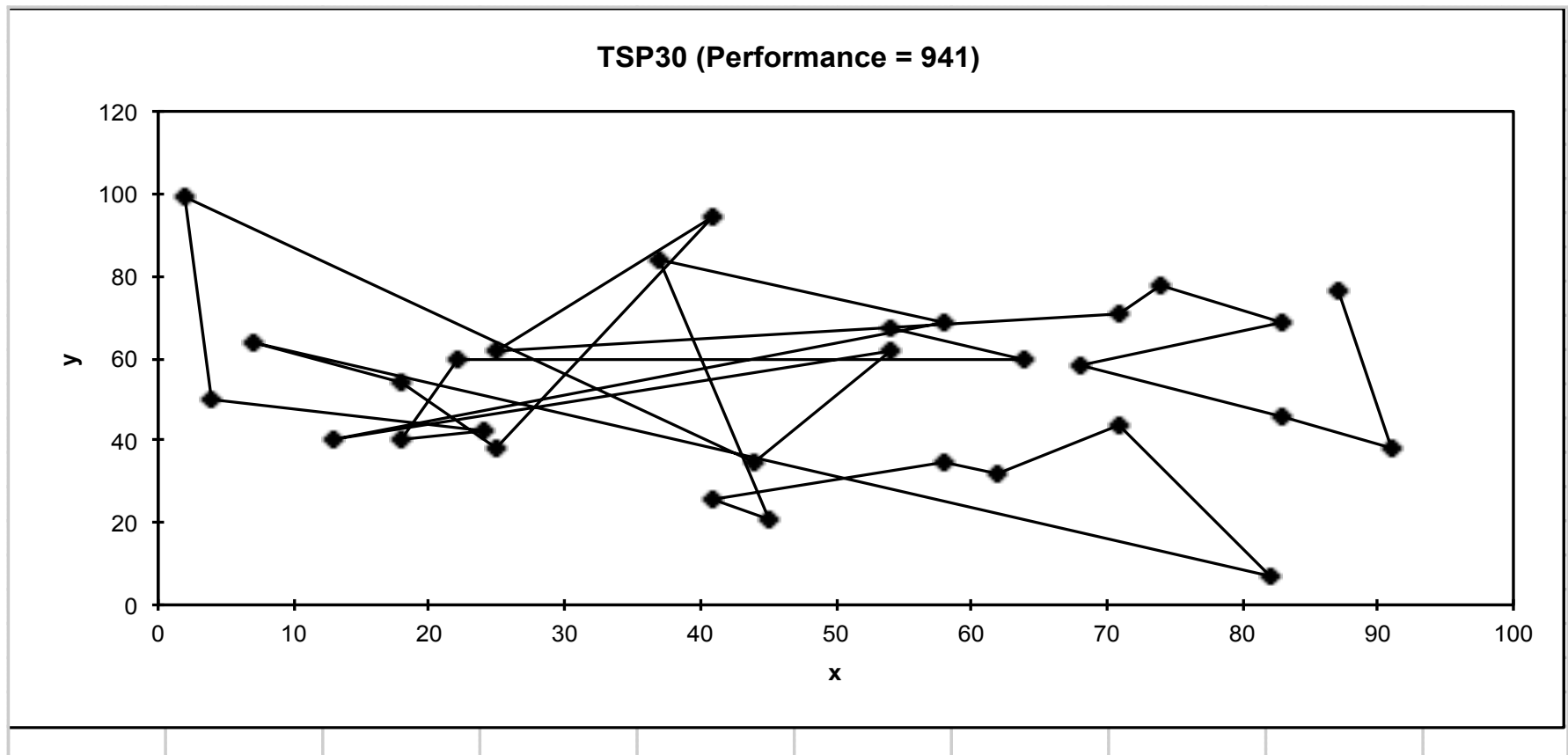
Mutation involves reordering of the list:



TSP Example: 30 Cities

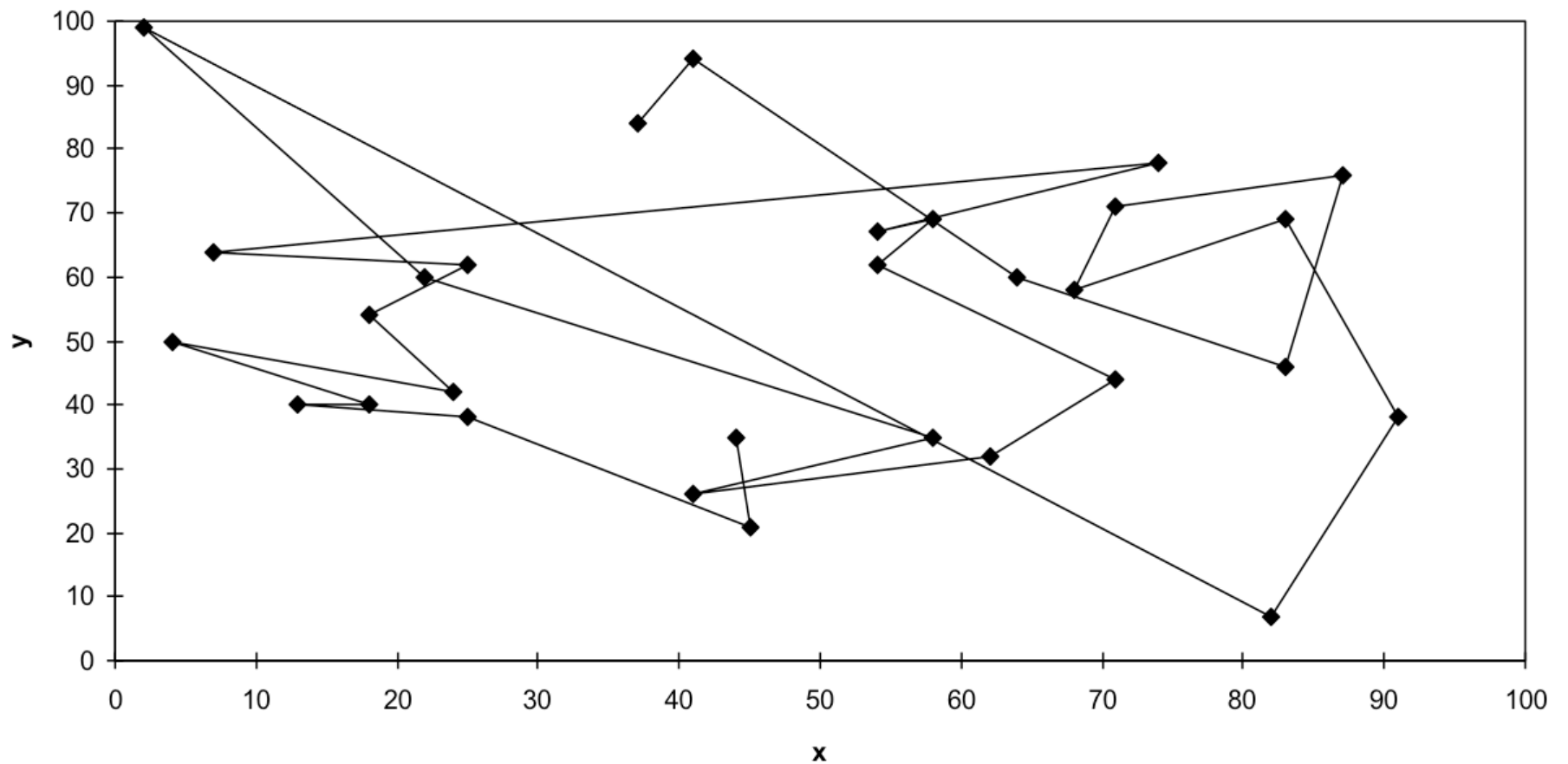


Solution _i (Distance = 941)

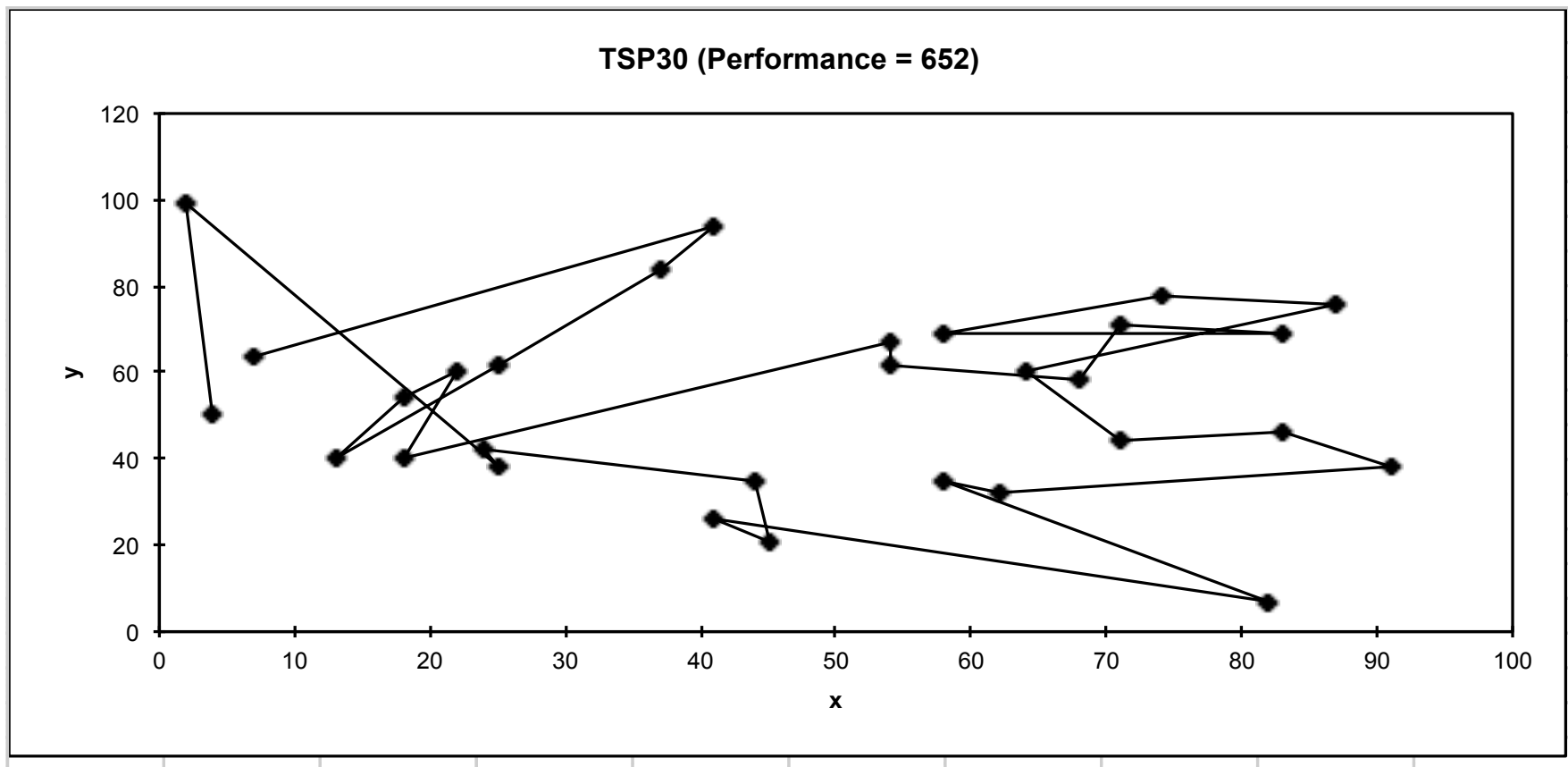


Solution j (Distance = 800)

TSP30 (Performance = 800)

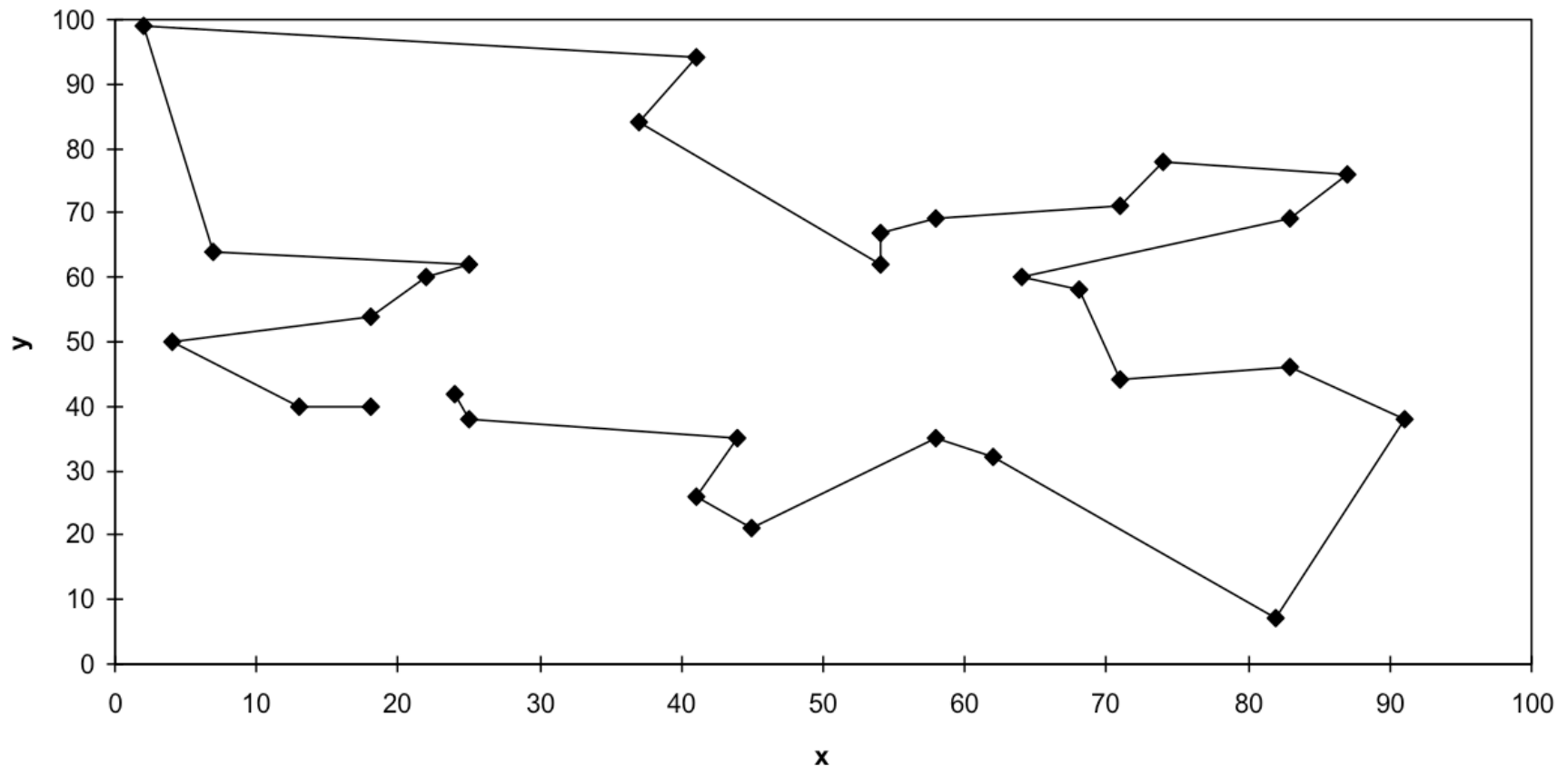


Solution $_k$ (Distance = 652)



Best Solution (Distance = 420)

TSP30 Solution (Performance = 420)



Overview of Performance

